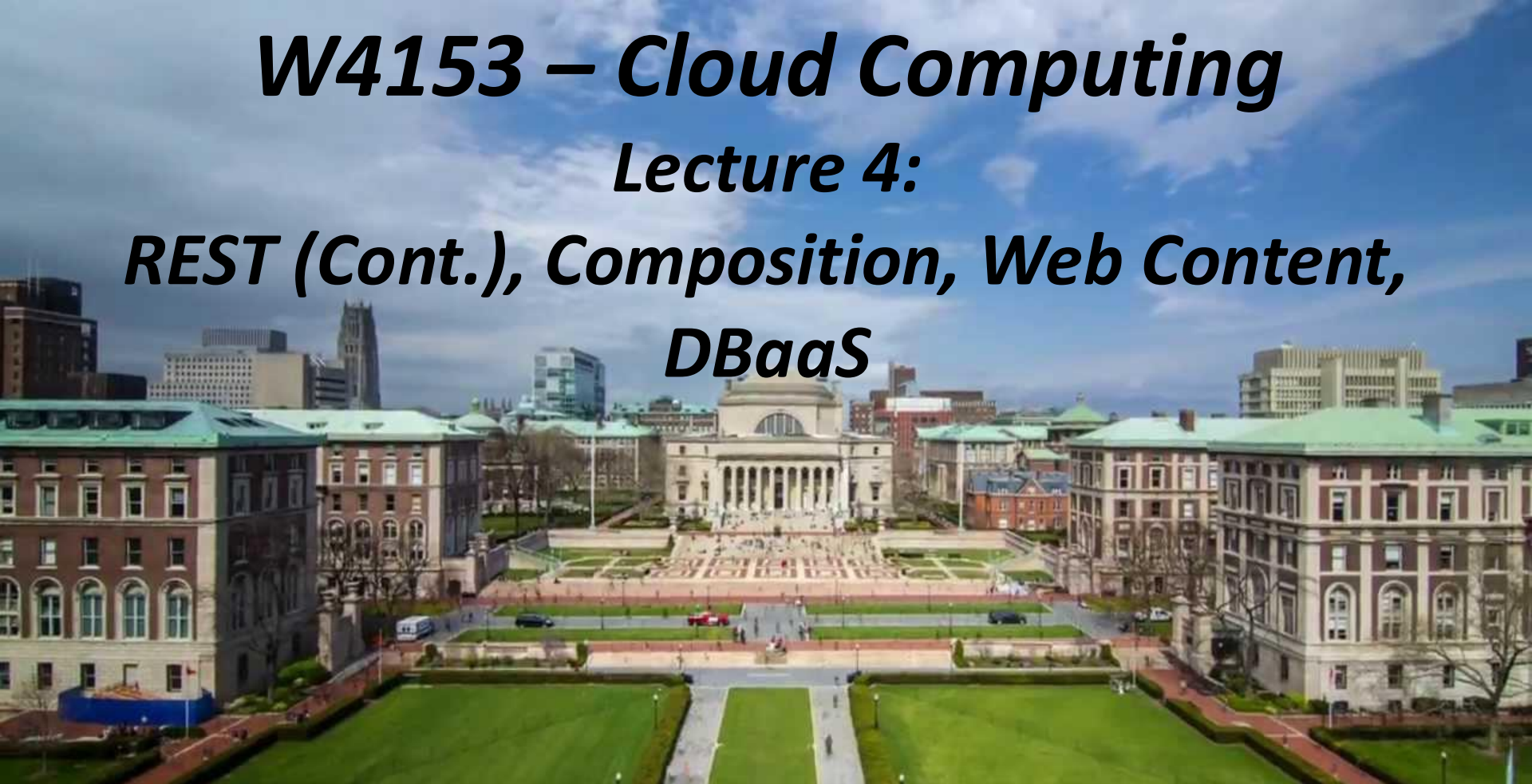




W4153 – Cloud Computing

Lecture 4:

REST (Cont.), Composition, Web Content, DBaaS

Contents

Contents

- Team presentations
- REST continued
 - Stateless
 - Layered System
 - Cacheable
 - eTag
- Function-as-a-Service
 - Concepts
 - My application scenario
 - Show code and walkthrough
- Service composition
 - Concepts
 - Orchestration, Choreography
 - Synchronous, Asynchronous
- Static Web Content
 - BLOB/Cloud storage
 - Websites
- DBaaS reminder and example
- 12 Factor Application Overview
- Next sprint preview

Team Presentations

Team Presentations and Updates

- Sri Bhuvana Vaishnavi Dasika
- Amrita Pal

REST Continued

REST (<https://restfulapi.net/>)

What is REST

- REST is acronym for **RE**presentational **S**tate **T**ransfer. It is architectural style for **distributed hypermedia systems** and was first presented by Roy Fielding in 2000 in his famous [dissertation](#).
- Like any other architectural style, REST also does have its own [6 guiding constraints](#) which must be satisfied if an interface needs to be referred as **RESTful**. These principles are listed below.

Guiding Principles of REST

- **Client-server** – By separating the user interface concerns from the data storage concerns, we improve the portability of the user interface across multiple platforms and improve scalability by simplifying the server components.
- **Stateless** – Each request from client to server must contain all of the information necessary to understand the request, and cannot take advantage of any stored context on the server. Session state is therefore kept entirely on the client.
- **Cacheable** – Cache constraints require that the data within a response to a request be implicitly or explicitly labeled as cacheable or non-cacheable. If a response is cacheable, then a client cache is given the right to reuse that response data for later, equivalent requests.
- **Uniform interface** – By applying the software engineering principle of generality to the component interface, the overall system architecture is simplified and the visibility of interactions is improved. In order to obtain a uniform interface, multiple architectural constraints are needed to guide the behavior of components. REST is defined by four interface constraints: identification of resources; manipulation of resources through representations; self-descriptive messages; and, hypermedia as the engine of application state.
- **Layered system** – The layered system style allows an architecture to be composed of hierarchical layers by constraining component behavior such that each component cannot “see” beyond the immediate layer with which they are interacting.
- **Code on demand (optional)** – REST allows client functionality to be extended by downloading and executing code in the form of applets or scripts. This simplifies clients by reducing the number of features required to be pre-implemented.

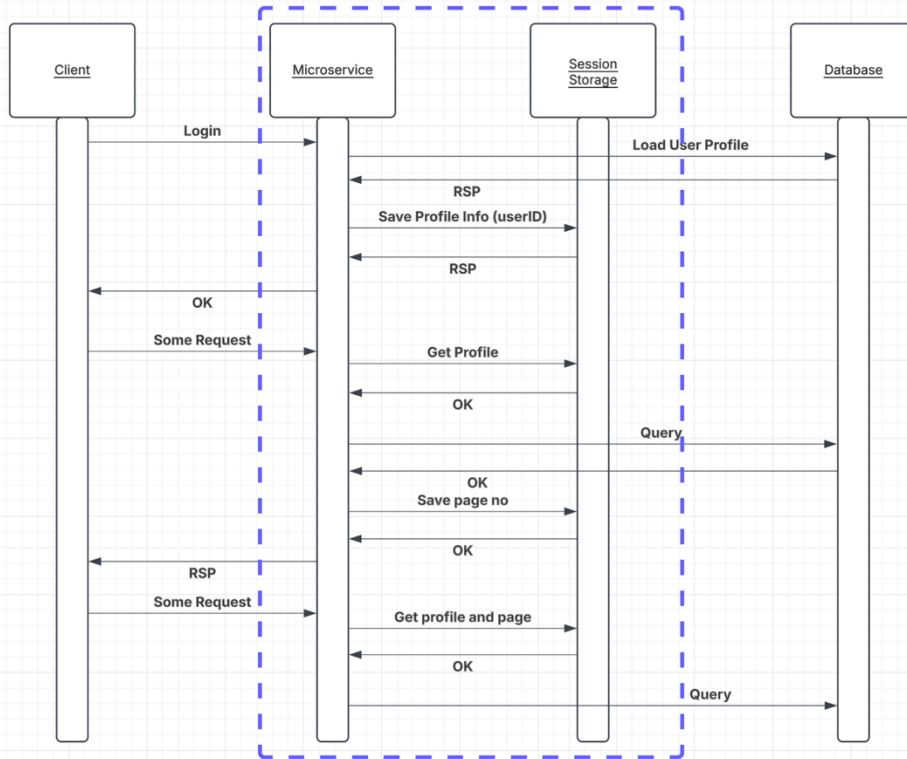
Stateless – Session State

- **Definition:** Session state refers to data associated with a specific user's interaction with an application over a period, typically from login to logout or until a defined timeout.
- **Scope:** It is confined to a single user's session and is not shared across different users or sessions.
- **Purpose:** It allows an application to maintain user-specific data (e.g., shopping cart contents, user preferences, form data) across multiple requests within that session, despite the stateless nature of protocols like HTTP.
- **Examples:** In web applications, session state might store items in a shopping cart, user authentication status, or temporary data needed for a multi-step form. Basically, it is history or context from the multiple exchanges between the client and server.
- **Persistence:** Session state is usually stored on the server-side (e.g., in memory, a database, or a dedicated session store) and linked to the client through a session ID (often stored in a cookie). It is generally considered ephemeral, meaning it can be lost if the server restarts or the session expires.

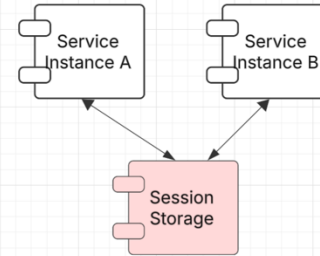
Stateless – Resource State

- **Definition:** Resource state refers to the actual data and configuration of resources managed by an application or system. These resources can be files, database records, cloud services, or any other persistent entities.
- **Scope:** Resource state is independent of individual user sessions and represents the current state of the resource itself. It is accessible and potentially modifiable by multiple users or processes.
- **Purpose:** It defines the persistent characteristics of a resource, allowing applications to interact with and manipulate these resources in a consistent manner.
- **Examples:** In a cloud environment, the state of an S3 bucket (e.g., its contents, access policies), a database record (e.g., user profile information), or a virtual machine (e.g., running, stopped) are all examples of resource state.
- **Persistence:** Resource state is designed for long-term persistence and survivability, typically stored in databases, file systems, or cloud storage, and remains even after client disconnects or application restarts.

Stateful Session

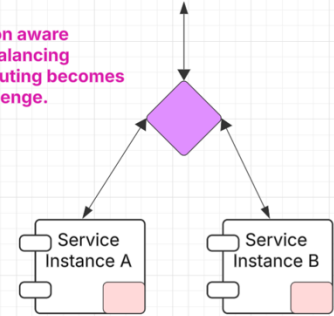


Session storage can be in the microservice's memory or in an external database/storage.



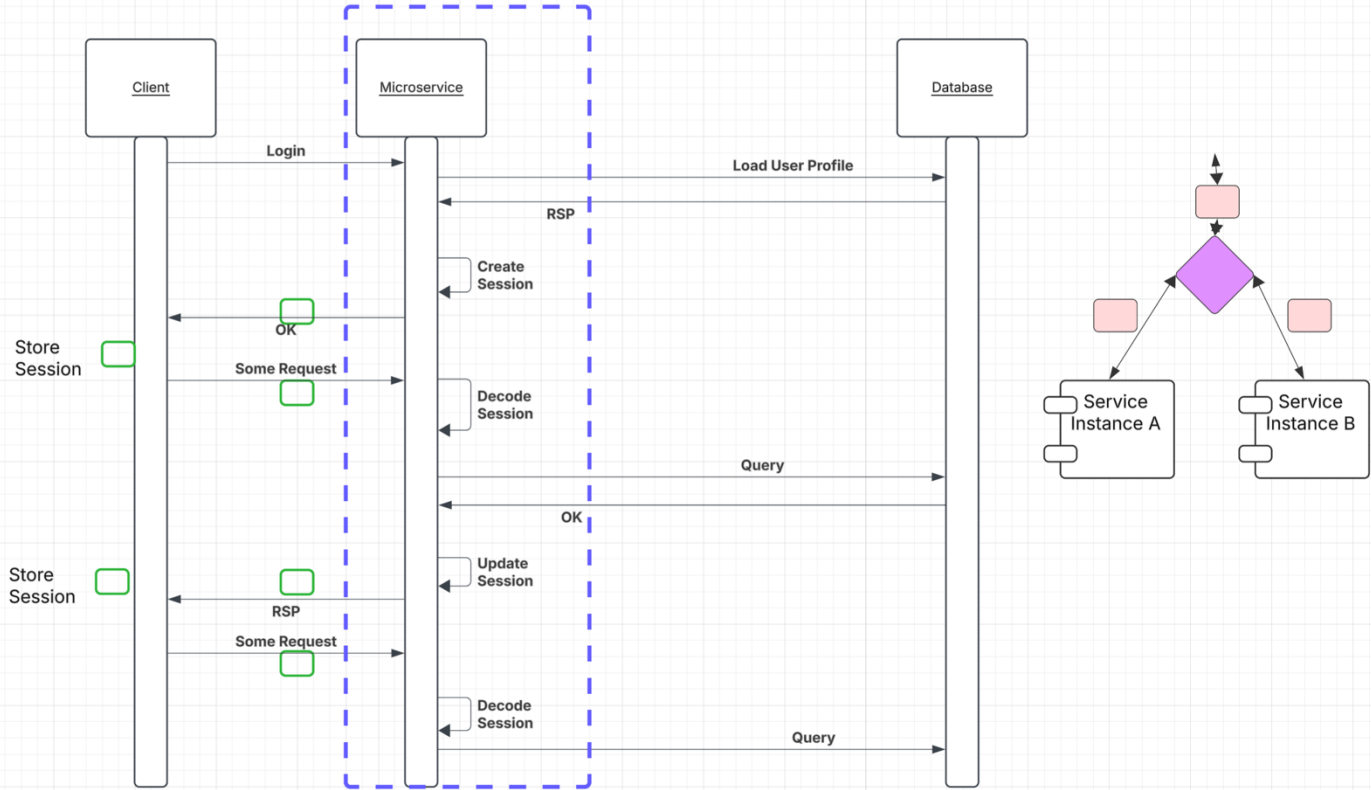
Session Storage becomes a scalability and availability challenge,

Session aware load balancing and routing becomes a challenge.



Stateless Session

- There are several places to store the session state on the client:
- If the client is a microservice, this is easy.
 - Browsers have several forms of application scoped storage
 - Local Storage
 - Session Storage
 - Cookies
 -



Session storage can be in the microservice's memory or in an external database/storage.

Idempotent

- From Wikipedia (<https://en.wikipedia.org/wiki/Idempotence>)
 - “Idempotence is the property of certain operations in mathematics and computer science whereby they can be applied multiple times without changing the result beyond the initial application.”
 - “In the Hypertext Transfer Protocol (HTTP), idempotence and safety are the major attributes that separate HTTP methods. Of the major HTTP methods, GET, PUT, and DELETE should be implemented in an idempotent manner according to the standard, but POST doesn't need to be. GET retrieves the state of a resource; PUT updates the state of a resource; and DELETE deletes a resource. As in the example above, reading data usually has no side effects, so it is idempotent (in fact nullipotent). Updating and deleting a given data are each usually idempotent as long as the request uniquely identifies the resource and only that resource again in the future.”
- Why does this matter? Ultimately, if I do a PUT and do not get an answer, my code has no way of distinguishing between:
 - The request got “lost” and was not processed.
 - The request was processed and the response got lost.
 - But more fundamentally, the network is flaky and loses messages, loses responses, retransmit messages,

Idempotent

<https://restfulapi.net/idempotent-rest-apis/>

Idempotency of HTTP Methods

HTTP Method	Idempotent	Safe
GET	✓	✓
HEAD	✓	✓
PUT	✓	✗
DELETE	✓	✗
POST	✗	✗
PATCH	✗	✗

- There are several techniques for ensuring correct implementation of idempotent processing.
- Examples:
 - Nonce (Idempotent Key)
 - Condition processing based on resources current state.
- Idempotent Key

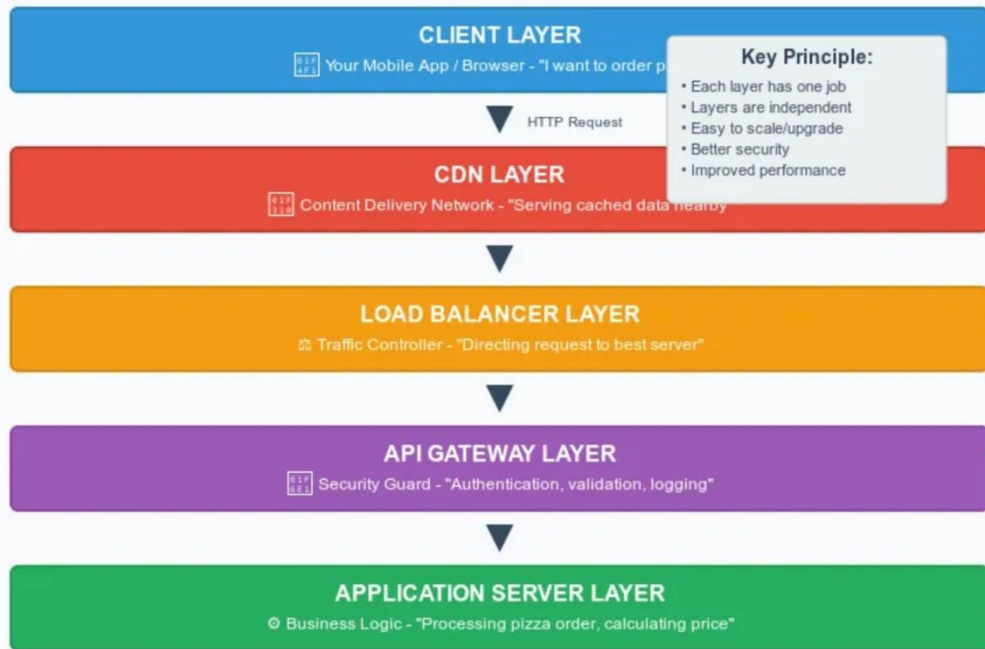
```
POST /payments
Content-Type: application/json
Idempotency-Key: 123e4567-e89b-12d3-a456-426614174000
```

```
{
  "amount": 100,
  "currency": "USD"
}
```

- Client generates the UUID.
- If the server has not processed the key, process and “remember” the response.
- If the server has processed the key, return the previous answer.

Layered System

REST Layered System Architecture



- REST client-server requests may and usually are processed by several layers or intermediaries in addition to the client and server application.
- You do not need to worry about this for now.
- The layers provide opportunities/locations to inject common "application logic."
- We will see examples later in the semester and use some examples.

Cacheable

- Caching can occur at several locations in the layering.
- Benefits
 - Improved Performance: Faster response times and reduced latency for clients.
 - Reduced Server Load: Less strain on databases and application servers.
 - Enhanced Scalability: Allows the API to handle a larger volume of requests.
 - Cost Savings: Can reduce infrastructure costs by minimizing the need for extensive scaling of primary data sources.
- Cacheable HTTP Methods:
 - GET requests: are inherently cacheable by default.
 - POST requests: are not cacheable by default but can be made cacheable by explicitly adding Expires or Cache-Control headers.
 - PUT and DELETE requests: are generally not cacheable as they modify resources.

Cacheable – Types of Caching

- Browser Caching: Clients (browsers) store responses for faster access to previously visited pages or resources.
- CDN Caching: Content Delivery Networks (CDNs) cache content at edge locations geographically closer to users, reducing latency and server load.
- Application-Level Caching: In-memory caches (e.g., Redis, Memcached) are used within the application to store frequently accessed data or computed results, avoiding repeated database queries or complex calculations.
- Request-Level Caching: Caching entire API responses for specific requests, particularly beneficial for read-heavy operations with predictable request patterns.

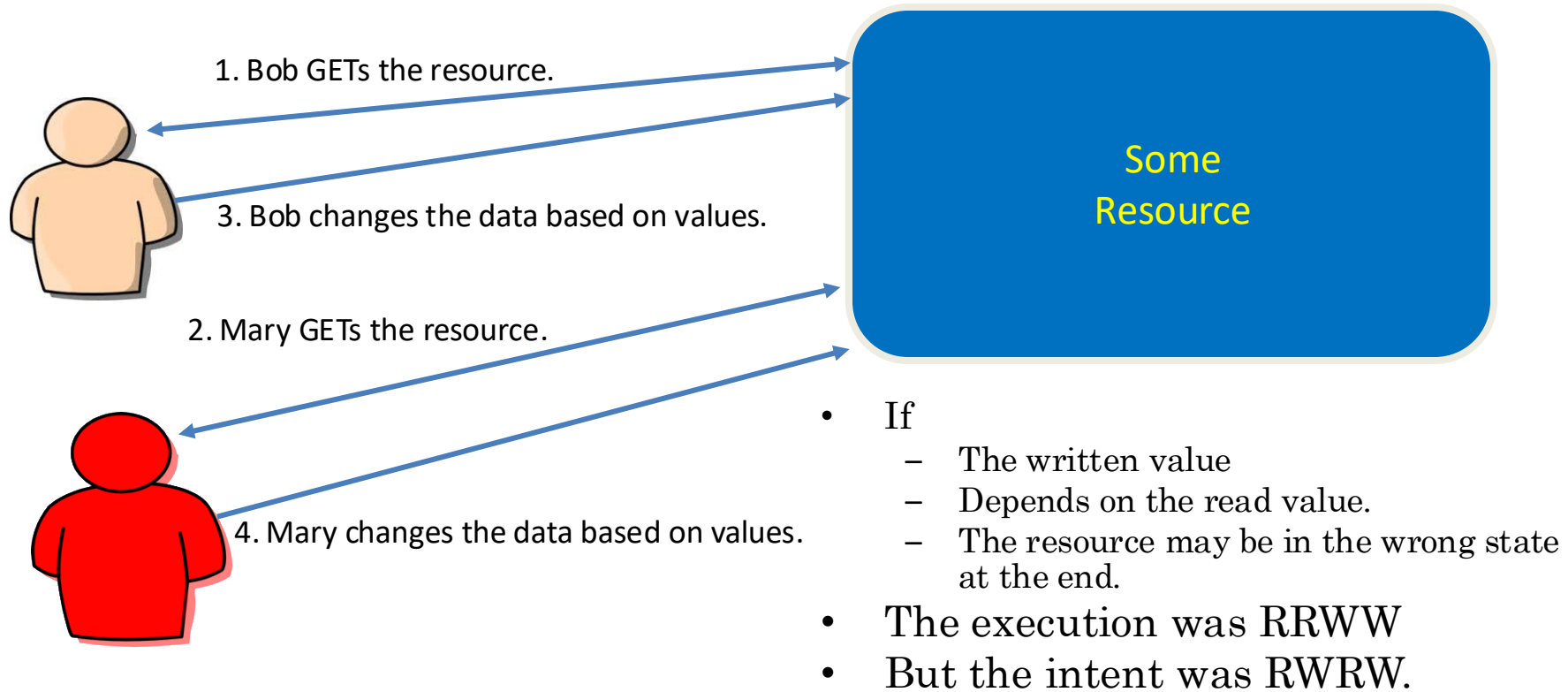
HTTP Caching Mechanisms

- **Cache-Control Header:** This header provides directives to control caching behavior for both client and intermediate caches. Common directives include public, private, no-cache, no-store, max-age, and s-maxage.
- **Expires Header:** Specifies a date and time after which the cached response is considered stale.
- **ETag (Entity Tag):** A unique identifier for a specific version of a resource. Clients can send the ETag in an If-None-Match header to check if the resource has changed. If it hasn't, the server can return a 304 Not Modified response, saving bandwidth.
- **Last-Modified Header:** Indicates the last modification date and time of a resource. Similar to ETag, clients can use If-Modified-Since to check for changes.

Cache Control Headers

- Cache-Control
 - public, max-age=3600 → cacheable for 1 hour
 - private → cacheable only in the client's browser
 - no-cache → must revalidate before reuse
 - no-store → do not cache at all
- Expires
 - Legacy header, absolute datetime when content becomes stale
 - Example: Expires Wed, 21 Oct 2025 07:28:00 GMT
 - Often replaced by Cache-Control: max-age
- Validation Headers
 - ETag (with If-None-Match)
 - Last-Modified (with If-Modified-Since)
- How They Work Together
 - First request → server returns resource with Cache-Control, ETag, Last-Modified
 - Subsequent request → client can use cached copy if still valid
 - Validation → If server sees If-None-Match matches ETag, responds 304 Not Modified (saves bandwidth and speeds response)

REST and Isolation: Read then Update Conflict



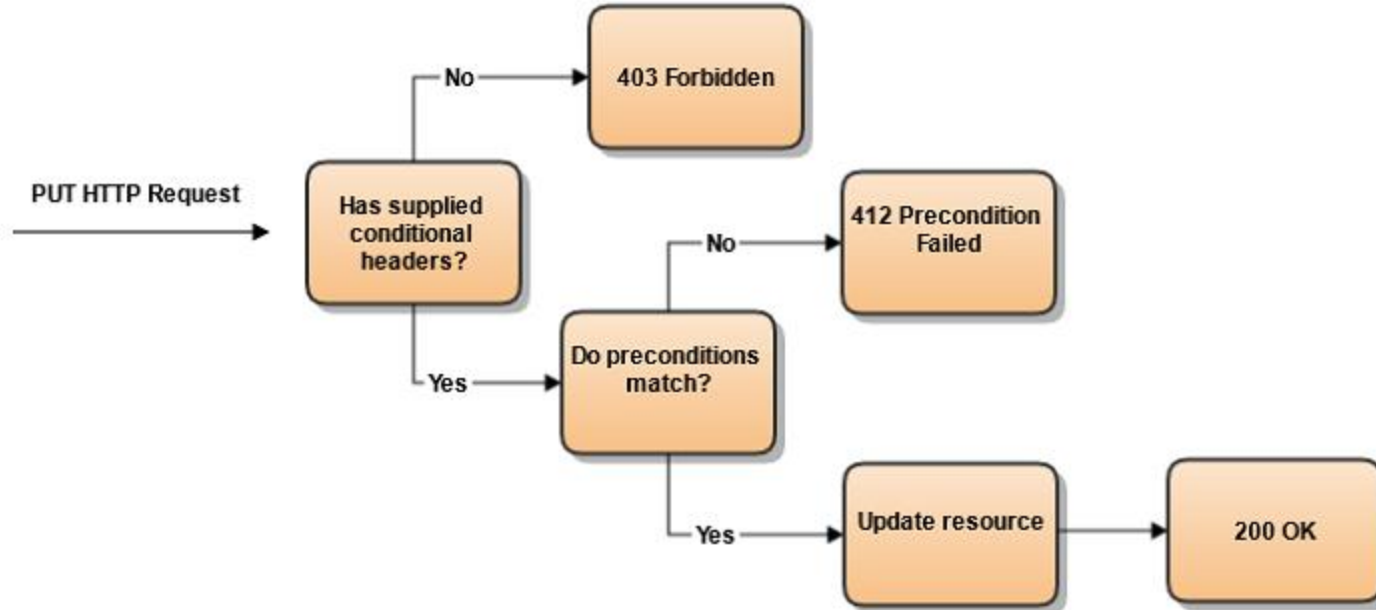
Isolation/Concurrency Control

This requires conversation state,
which would violate the REST
Stateless principle.

- There are two basic approaches to implementing isolation
 - Locking/Pessimistic, e.g. cursor isolation
 - Optimistic: Before committing, each transaction verifies that no other transaction has modified the data it has read.
- How does this work in REST?
 - The server maintains an ETag (Entity Tag) for each resource. **Or computes using hashing.**
 - Every time a resource's state changes, the server computes a new ETag.
 - The server includes the ETag in the header when returning data to the client.
 - The server may *optionally* maintain and return "Last Modified" time for resources.
- Semantics on updates
 - If-Match – value of a previous calls ETag response used with a PUT or DELETE. Server should only act if nobody else has modified the resource since you last fetched it. Otherwise provides a 412.
 - If-Modified-Since – value of a previous Last-Modified response used with a GET. Server should only provide a response if the resource was modified since the timestamp submitted. Use in conjunction with If-None-Match in case the change occurred within the same second. Otherwise provide a 304.
 - If-None-Match – value of a previous calls ETag response used with a GET. Server should only provide a response if the ETag doesn't match, i.e. the resource has been altered. Otherwise provide a 304.
 - If-Unmodified-Since – value of a previous Last-Modified response used with a PUT or DELETE. Server should only act if nobody else has modified the resource since you last fetched it. Otherwise provides a 412

Conditional Processing

(<https://blog.4psa.com/rest-best-practices-managing-concurrent-updates/>)

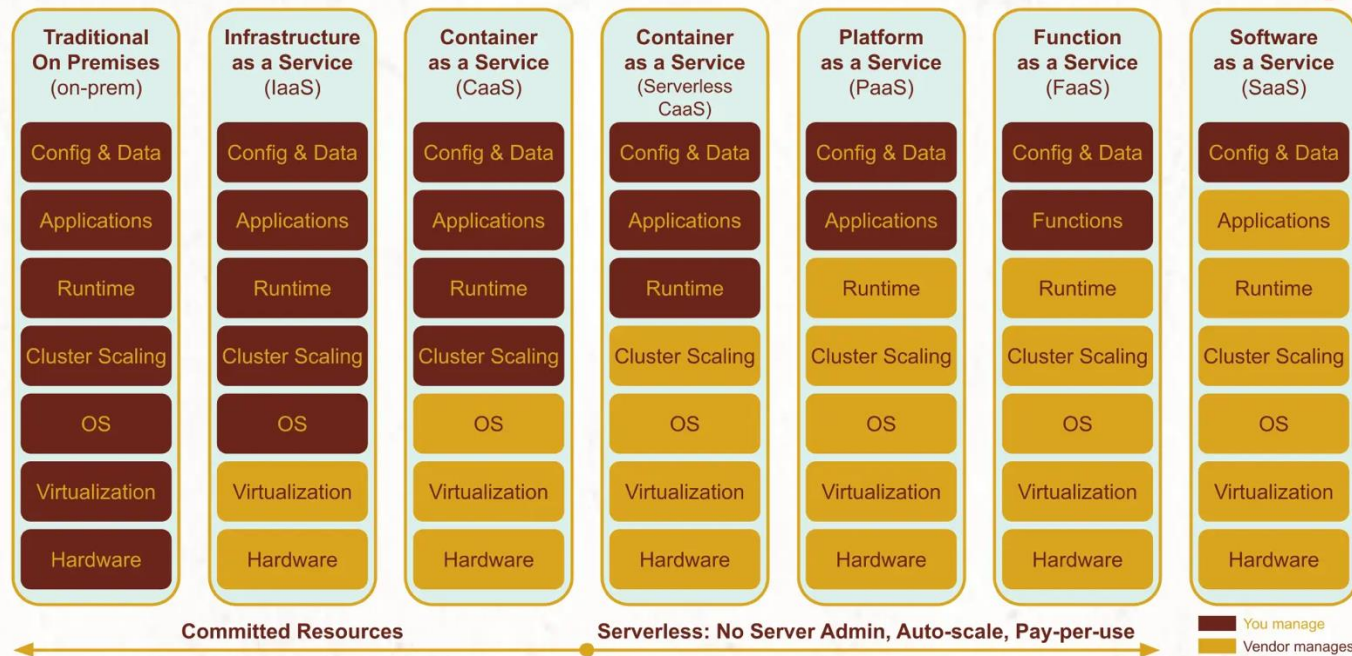


Function-as-a-Service

Cloud Layers

Cloud Deployment Spectrum

ml4devs.com/serverless 



 © Satish Chandra Gupta, Creative Commons BY-NC-ND 4.0 International License.

scgupta 

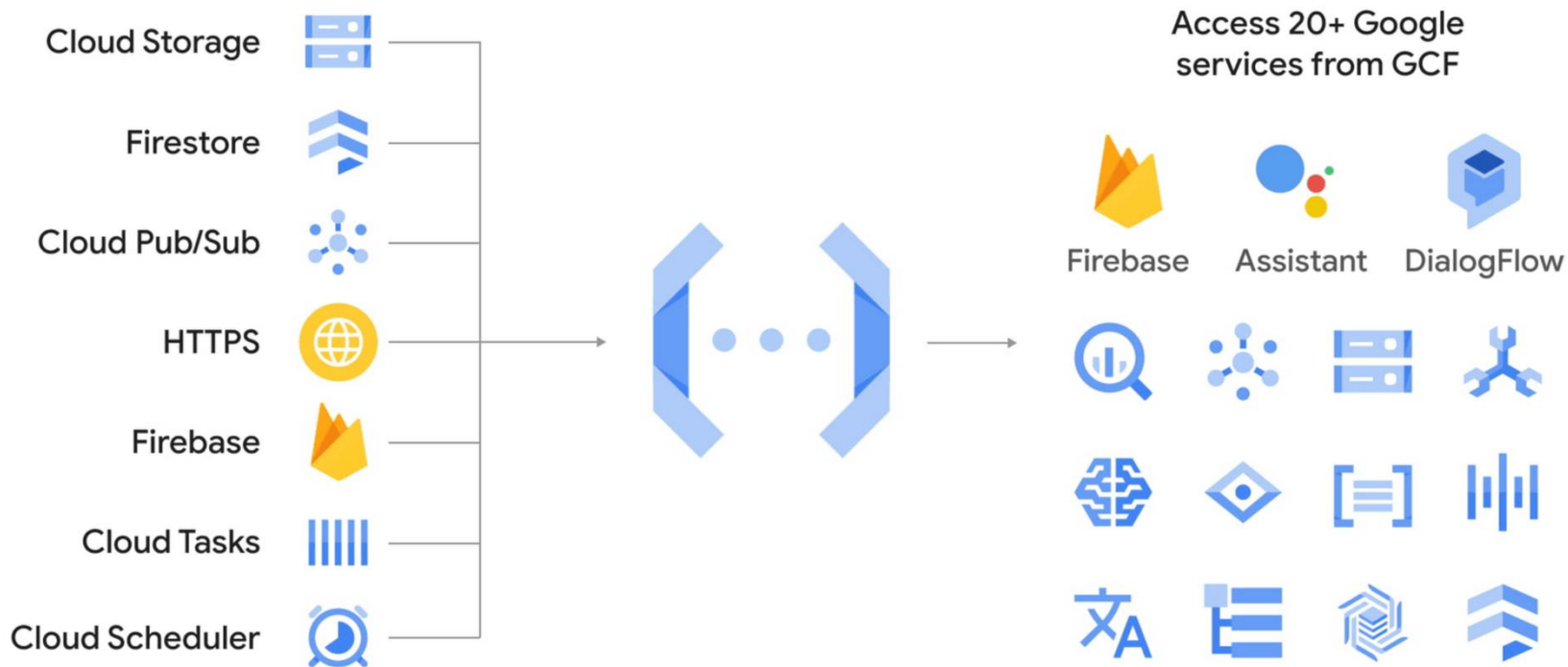
linkedin.com/in/scgupta 

Function-as-a-Service Principles

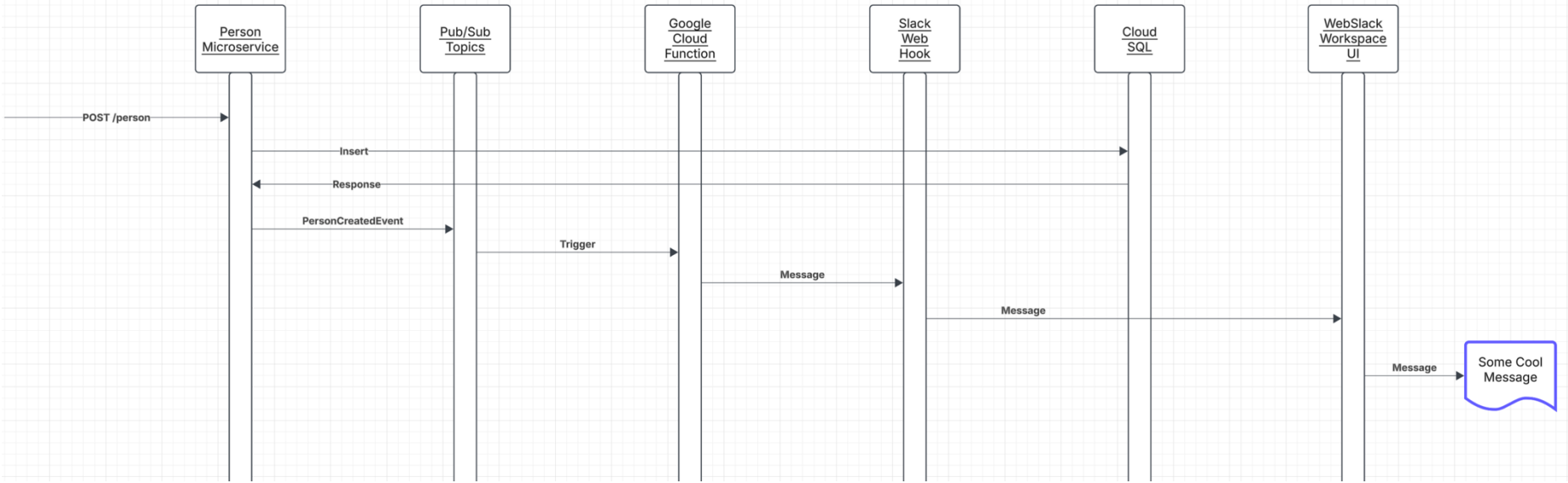
- Event-Driven Execution:
 - Specific events trigger FaaS functions. These events can be an HTTP request, a message in a queue, database update, scheduled task,
 - The corresponding function executes when an event occurs.
 - Think of this being like the generalization of a database trigger.
- Stateless Functions: Each function is self-contained. It maintains no state between invocations, enabling easy scaling and parallel execution.
- Short-Lived Execution: Functions run for a short duration, typically seconds to minutes. The system creates them when needed and removes them once their task is complete, optimizing resource use.
- Managed Infrastructure: The cloud provider handles the underlying infrastructure, including servers, scaling, and security. It abstracts the functional complexities away from the developers.

Google Cloud Functions

Cloud Functions: the glue that connects cloud services



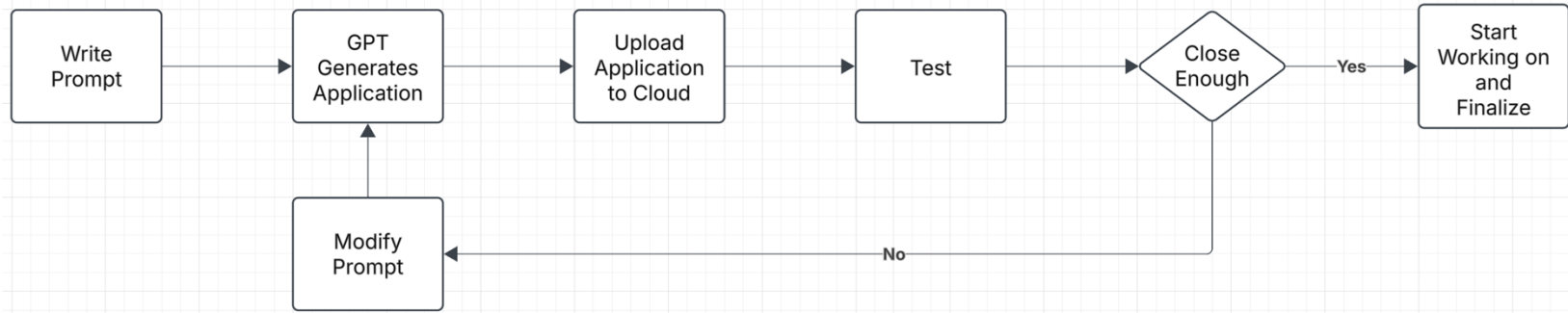
Scenario



- For now, just focus on plumbing everything together.
- You do not need to connect the microservice. You can just trigger with gcloud CLI.
- Collaboration/Social Media space:
 - I will use either Slack or Discord. You can choose whatever you want.
 - For now, you can just write to the Google log to check and demonstrate that your function ran.

Project Walkthrough

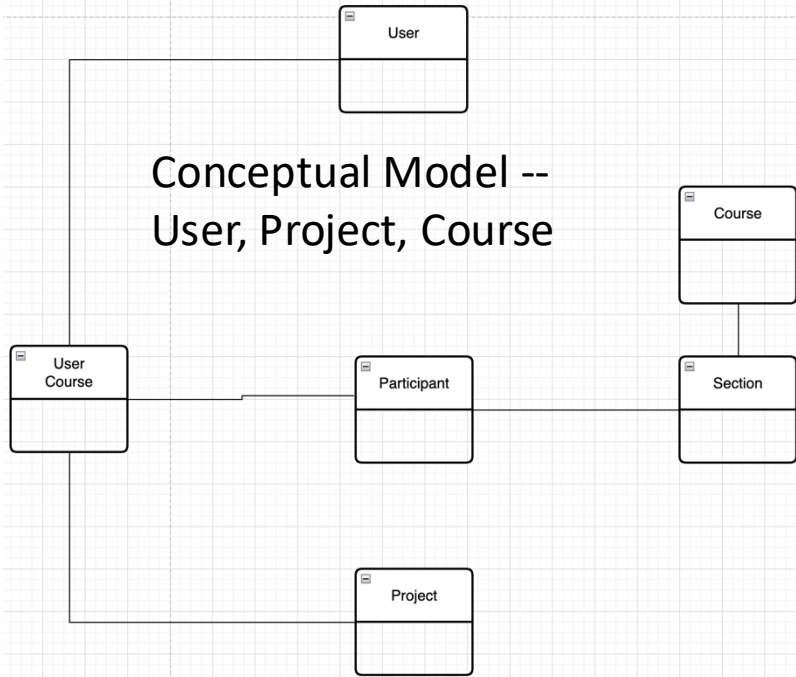
- `/Users/donald.ferguson/Dropbox/000/000-A-2025-Repo/simple_html`
- Not tested on GCP yet. Will do this weekend, but ...
 - This is a simple ChatGPT generated template/starter project.
 - The new style of programming is ...



- You can/should do the same.

Composition

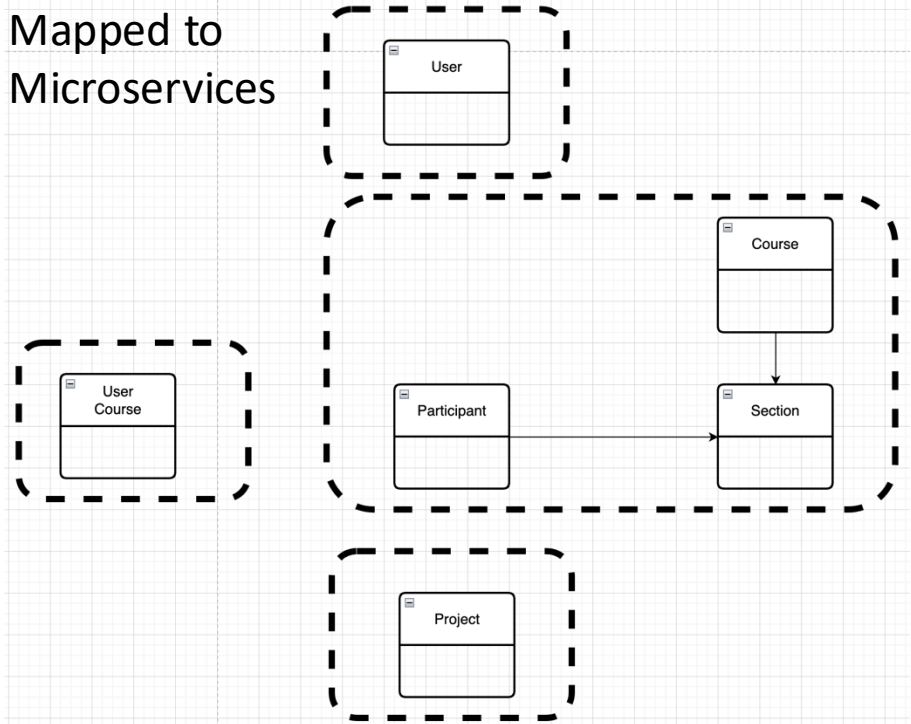
Resource Oriented and Microservices – My Project



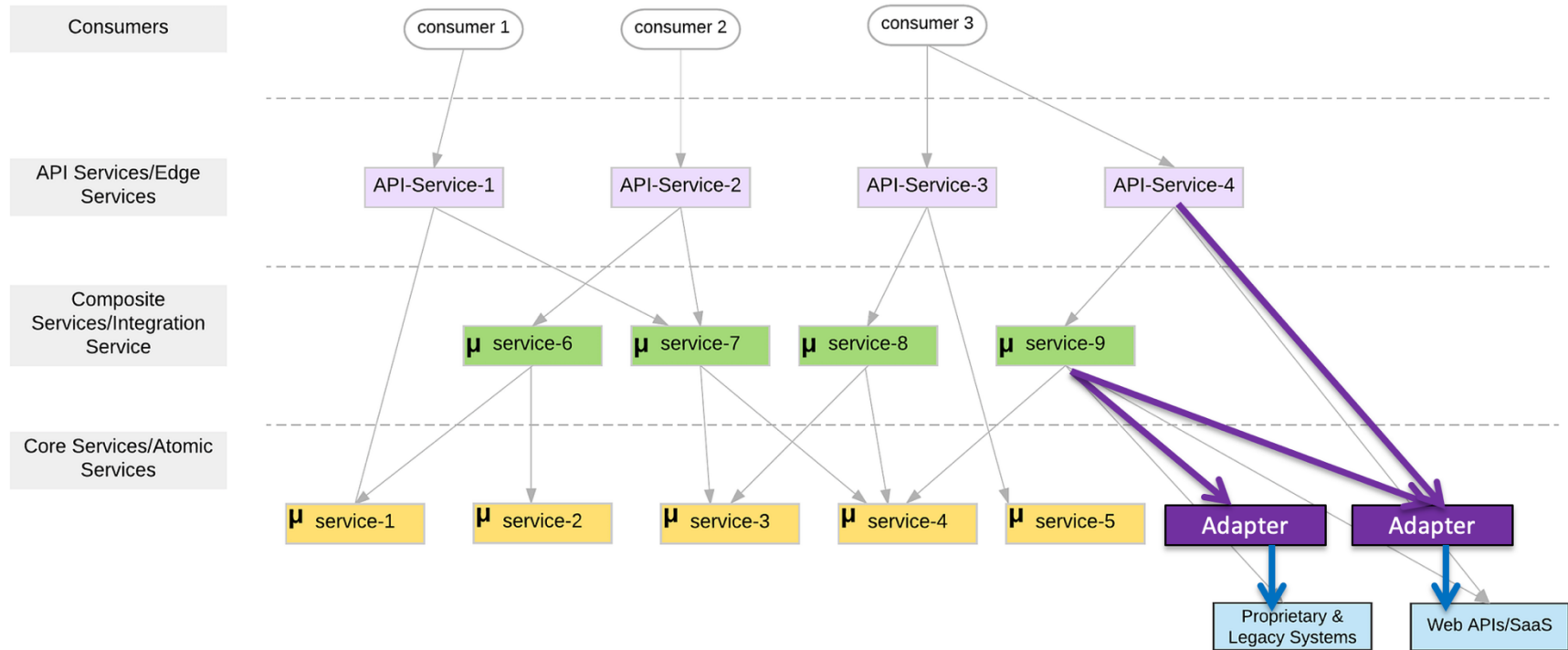
This creates some interesting challenges, e.g.

- Joins
- Foreign Keys

Mapped to
Microservices



If the cluster of resources might be useful
in other solutions → It may be a microservice.



First Group of Microservices

- My first sprint will have 5 microservices:

- 5 core/atomic microservices:

- user_info
- courseworks_adaptor
- course_info
- student_coupons
- project_teams

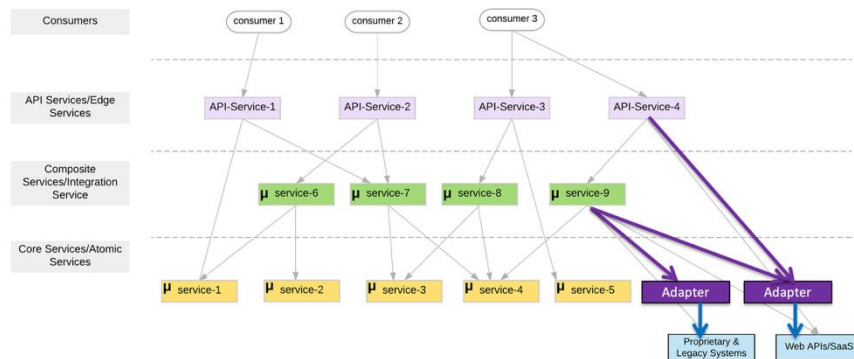
- 1 composite/integration microservice: core_course_management

- Integration with Slack, Discord or

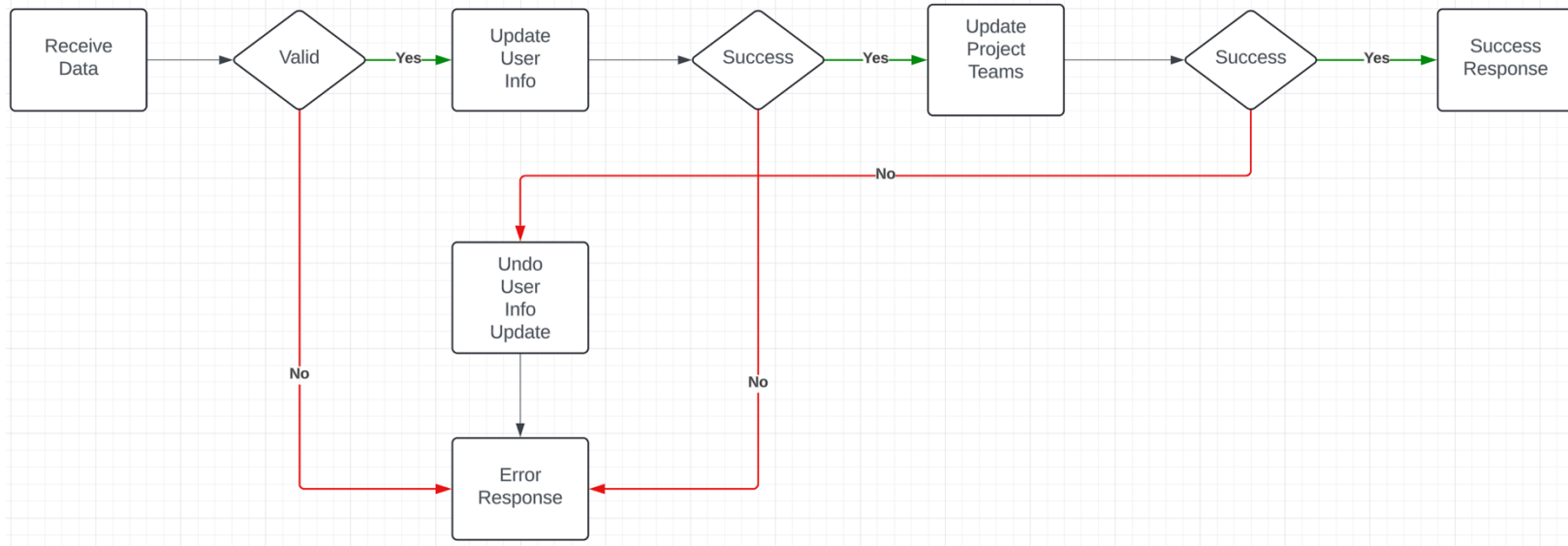
- The composite service:

- Implements “consistency” in the data in the core services, e.g. “foreign keys.”
- Eliminates the need for complex logic in the UI or client, which would be impacted by changes.
- Exposes only information from the basic services that the client requires.

- The composite service is conceptually similar to a database view.

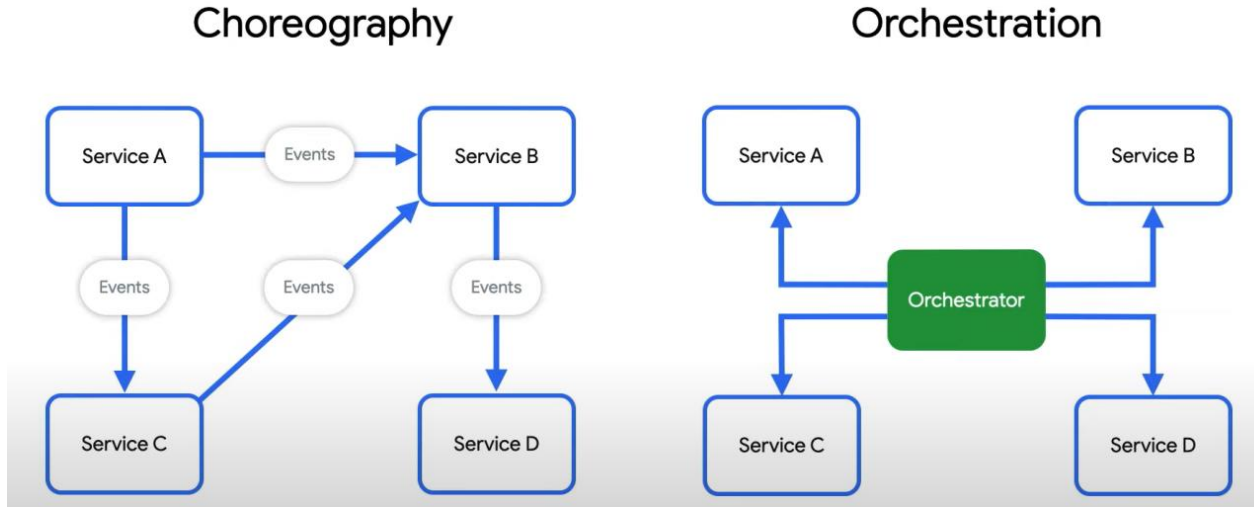


Composite Microservice Logic



- The GET on the composite invokes GET on several atomic services. This can occur synchronously or asynchronously in parallel.
- PUT, POST and DELETE are more complex. Your logic may have to undo some delegated updates that occur if others fail.
- To get started, we will just use simple logic but explore more complex approaches.

Behavioral Composition



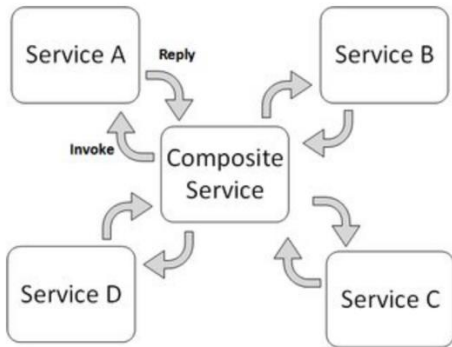
- There are two base patterns: choreography and orchestration.
- There also several different implementation technologies and choices.
- And many related concepts, e.g. Sagas, Event Driven Architectures, Pub/Sub,
- We will cover and explore incrementally, but for now we will keep it simple and do traditional “if ... then ...” code.

Concepts

Service orchestration

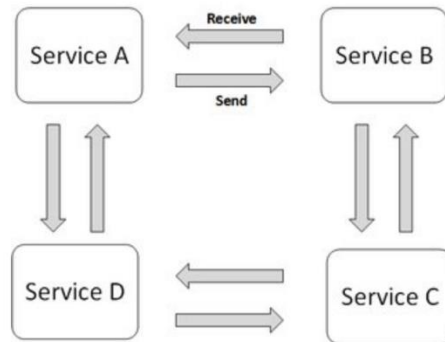
Service orchestration represents a single centralized executable business process (the orchestrator) that coordinates the interaction among different services. The orchestrator is responsible for invoking and combining the services.

The relationship between all the participating services are described by a single endpoint (i.e. the composite service). The orchestration includes the management of transactions between individual services. Orchestration employs a centralized approach for service composition.



Service Choreography

Service choreography is a global description of the participating services, which is defined by exchange of messages, rules of interaction and agreements between two or more endpoints. Choreography employs a decentralized approach for service composition.



The choreography describes the interactions between multiple services, where as orchestration represents control from one party's perspective. This means that a **choreography** differs from an **orchestration** with respect to where the logic that controls the interactions between the services involved should reside.

These are not formally, rigorously defined terms.

In Code: Composition and Asynchronous Method Invocation

- Composite operations can “take a while.”
- “Holding open connections and waiting can be bad.”
 - Consumes resources.
 - Connections can break.
 -

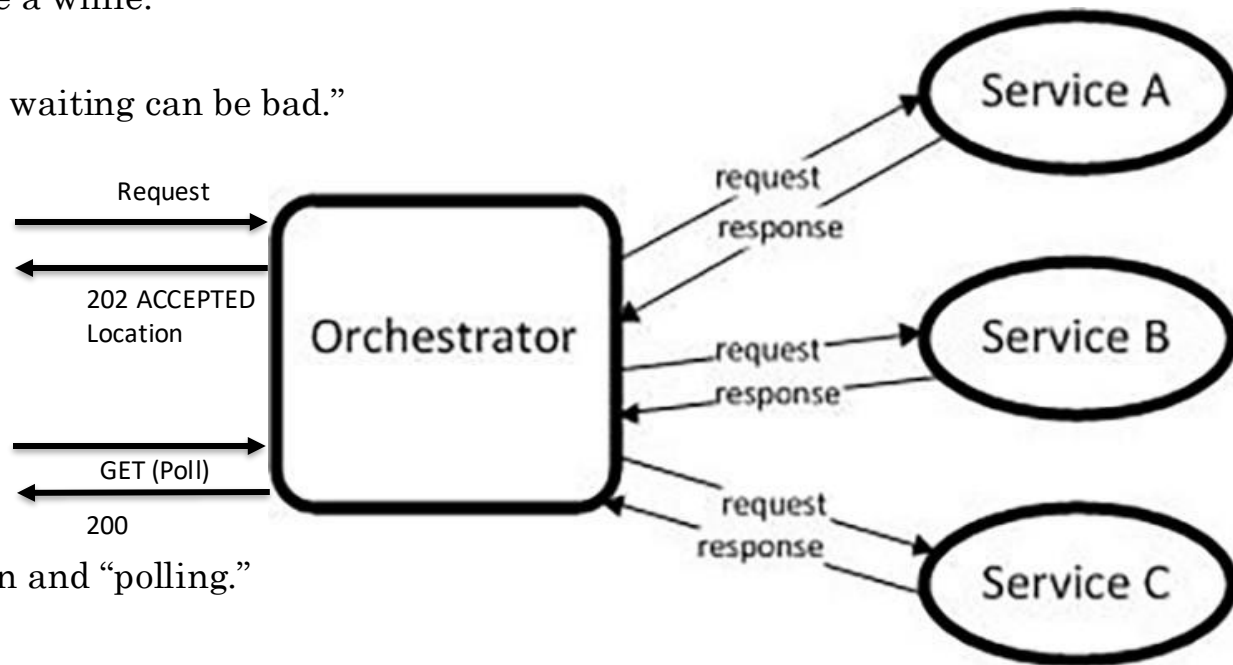
- There are several options:

- Polling
- Callbacks
- Asynchronous calls

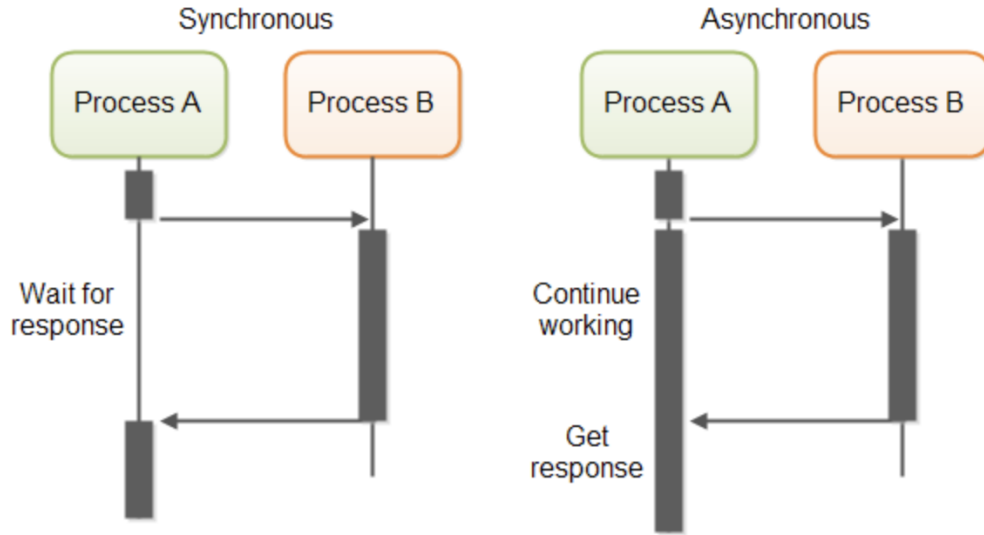
- Our next pattern is composition and “polling.”

Create a new user requires:

- Updating the user database.
- Verifying a submitted address.
- Creating an account in the catalog service.



In Code: Service Orchestration/Composition

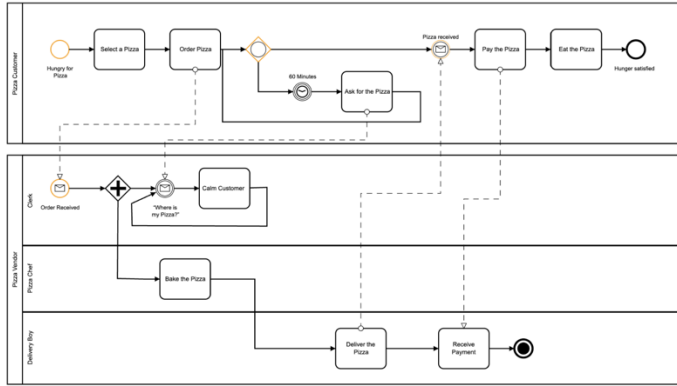


RESTful HTTP calls can be implemented in both a synchronous and asynchronous fashion at an IO level.

- One call to a composite service
 - May drive calls to multiple other services.
 - Which in turn, may drive multiple calls to other services.
- Synchronous (Wait) and calling one at a time, in order is
 - Inefficient
 - Fragile
 -
- Asynchronous has benefits but can result in complex code.

Service Orchestration

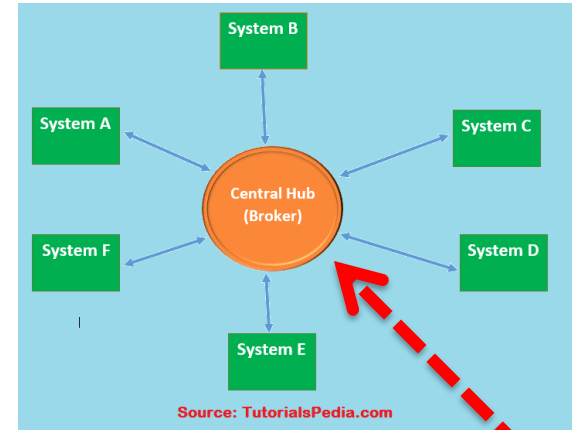
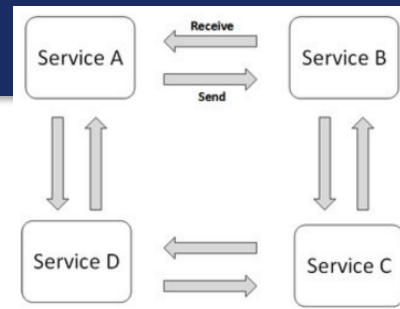
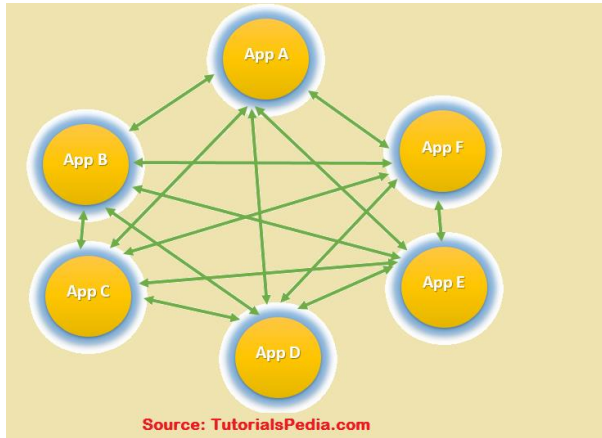
- Implementing a complex orchestration in code can become complex.
- High layer abstractions and tools have emerged, e.g. BPMN.
 - “Business Process Model and Notation (BPMN) is a graphical representation for specifying business processes in a business process model.”  [expand in new window](#)



- There are products for defining BPMN processes, generating code, executing,, e.g. Camunda: <https://camunda.com/bpmn/>
- There are other languages, tools, execution engines,

Choreography

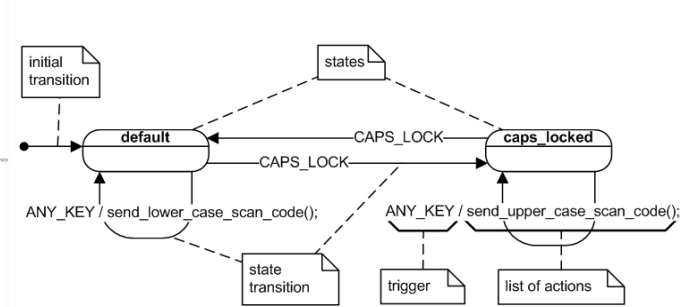
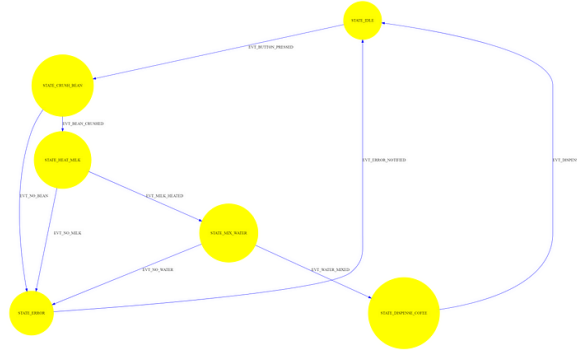
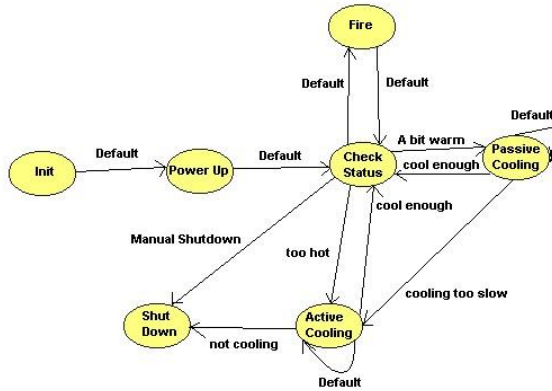
- We saw the basic diagram, but ... this is an anti-pattern and does not scale.



- But, how do you write the event driven microservices?
 - Well, you can just write code
 - Or use a state machines abstraction.

Pub/Sub and EDA
Which we will cover.

State Machine



- A state machine is an abstraction.
- There are many models for defining state machines.
- There are also development tools, runtimes,
- We will look at a specific example: AWS Step Functions

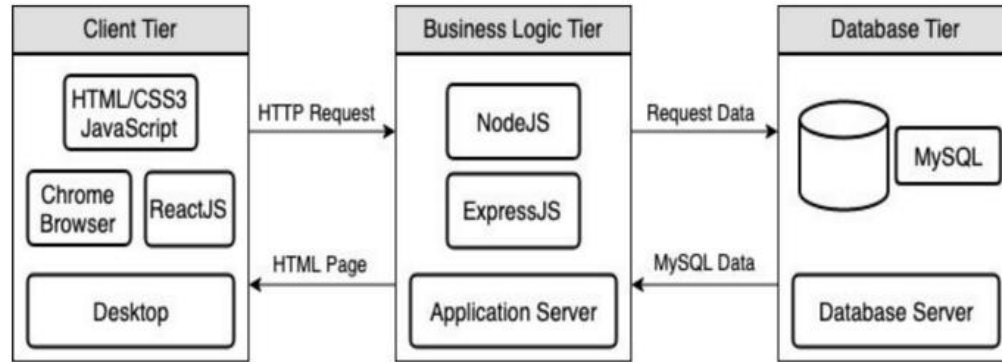
Show Code

- Show example of calling URL synchronously and asynchronously
 - `/Users/donald.ferguson/Dropbox/000/000-A-2025-Repos/sync_async/async1.py`
 - `/Users/donald.ferguson/Dropbox/000/000-A-2025-Repos/sync_async/sync1.py`
- One of the steps in your next sprint is:
 - Build a composite microservice that aggregates your basic microservices.
 - Demonstrating one method on the composite aggregating the base services synchronously.
 - Demonstrating one method on the composite aggregating the base services asynchronously.
- You are probably confused as heck right now.
 - Think of your initial microservices as base tables.
 - Think of the composite as a VIEW representing a JOIN of the base tables.
 - Your composite microservice's code implements the basic operations, and perhaps some additional logic, in the JOIN to “compose” that basic services.

Web Content (BLOB Storage)

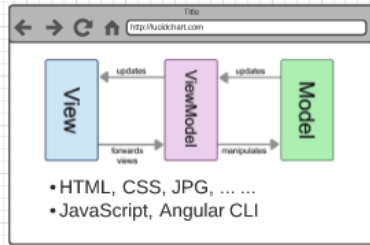
Full Stack Web Application – Reminder

<https://levelup.gitconnected.com/a-complete-guide-build-a-scalable-3-tier-architecture-with-mern-stack-es6-ca129d7df805>

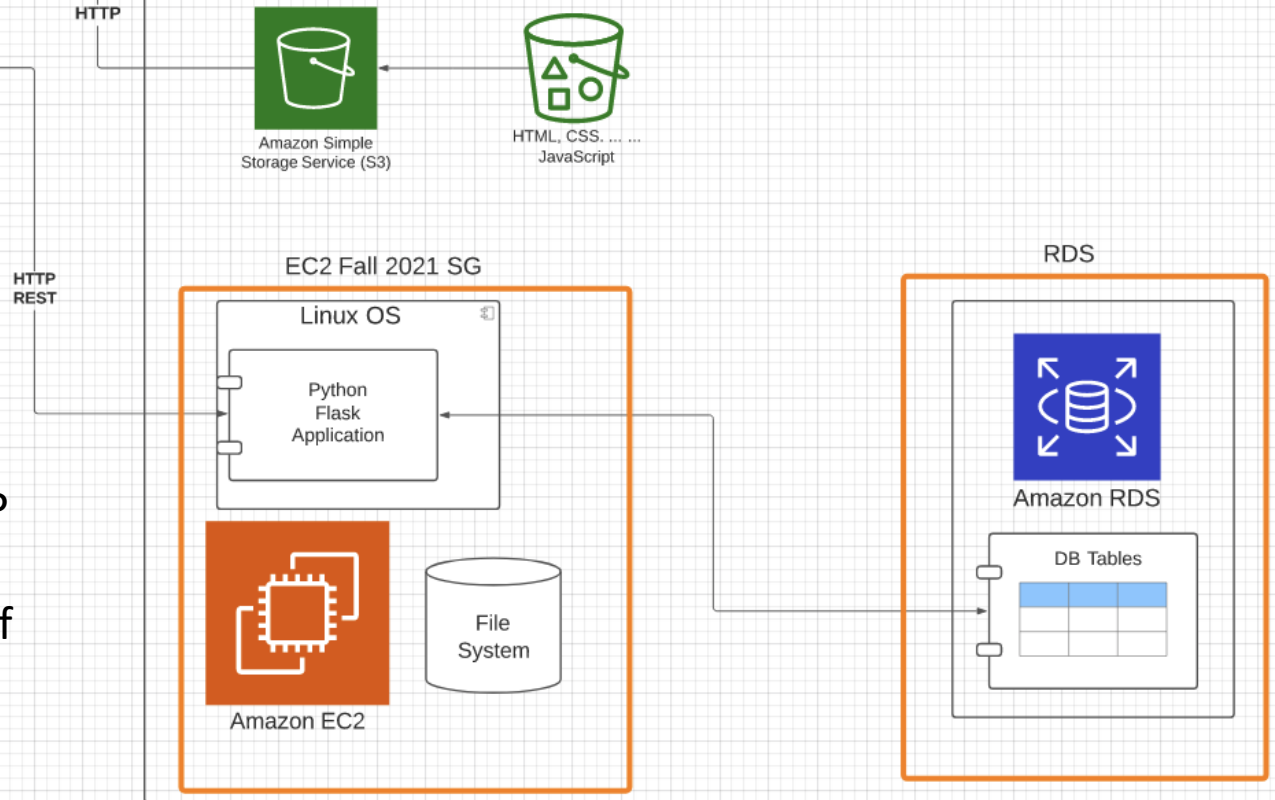


- We have seen how to use Flask (or FastAPI) to implement an endpoint (a set of paths):
 - Some of the paths can run application logic and return the result data.
 - Some can return pages that a browser can render. Basically, these paths are returning the “client application” to the browser to execute, and may have API calls in JavaScript/Typescript.
 - Do not worry about this now.
- But, running application logic and simply delivering files are two different “concerns.”
 - Having one runtime implement both would violate the SOLID principle.
 - The design points are different: “You can be a floor wax or a mouth wash, but not both.”
- So, we need a separate “web server.”

Browser



Amazon Web Services



- The structure on GCP is similar.
- I am in the process of “porting” from AWS to GCO.

BLOB: Binary Large Object

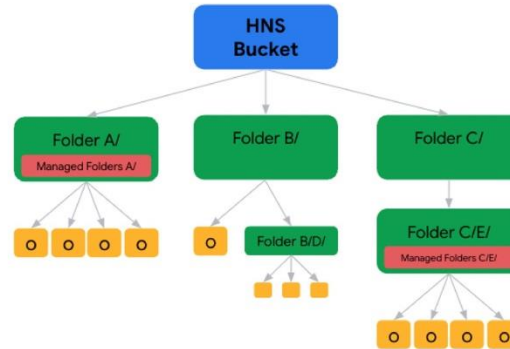
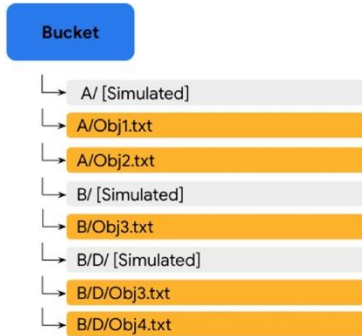
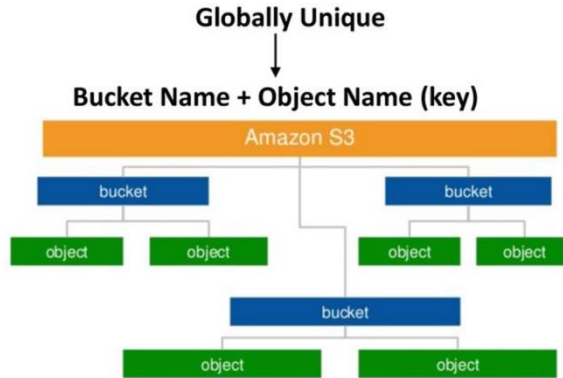
- Definitions:
 - “A binary large object (BLOB or blob) is a collection of binary data stored as a single entity. Blobs are typically images, audio or other multimedia objects, though sometimes binary executable code is stored as a blob. They can exist as persistent values inside some databases or version control system, or exist at runtime as program variables in some programming languages.”
(https://en.wikipedia.org/wiki/Binary_large_object)
 - “Blob is the data type in MySQL that helps us store the object in the binary format. It is most typically used to store the files, images, etc media files for security reasons or some other purpose in MySQL.” (<https://www.educba.com/mysql-blob/>)
- Intent:
 - File systems are not good for many application scenarios, which is why database management systems emerged.
 - Database management systems implement structured (or semi-structured) data, e.g. tables.
 - Database BLOBs were a way to have some “big things” as table or document properties.

Amazon S3



We used the equivalent capability on Azure for uploading CAD files, storing processed files and results, etc.

Concepts of S3 – Namespace



Can support a logical hierarchy with a convention on keys.

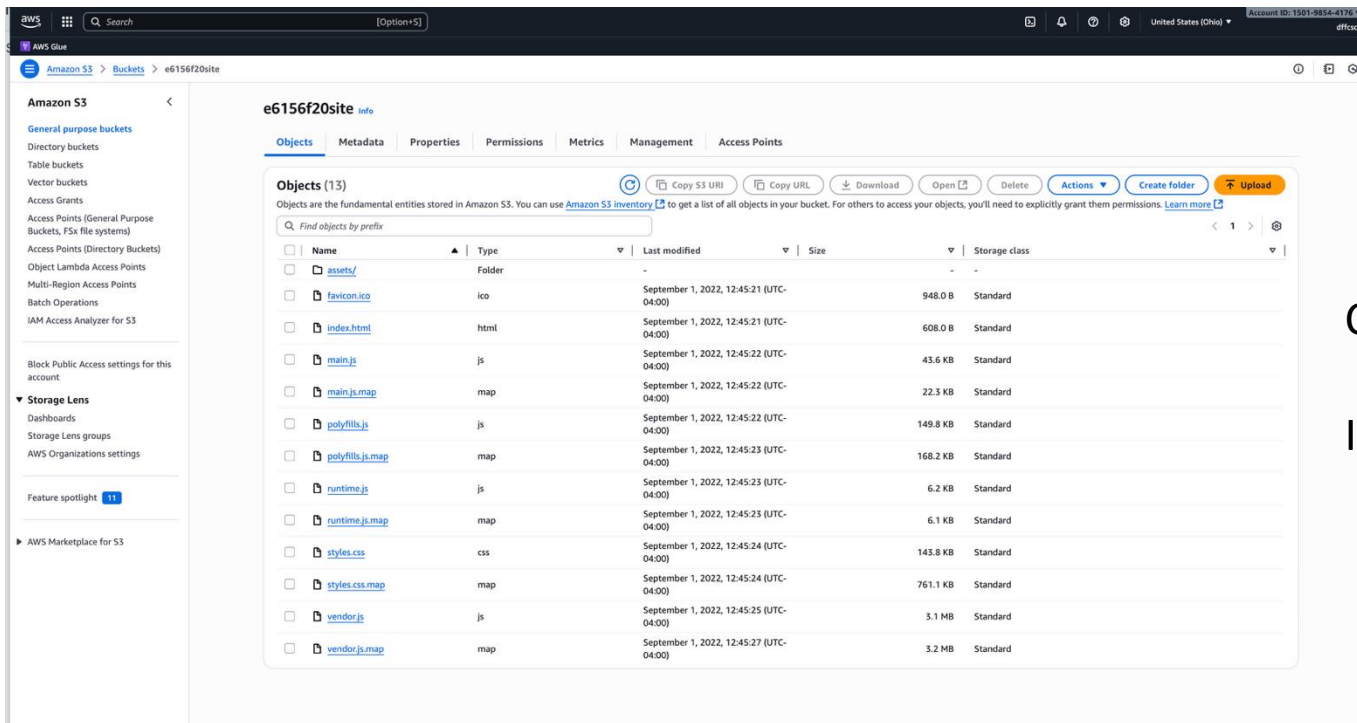
- x/y
- x/z
- x/y/z
-

Acts kind of file system like, but is really just key strings.

GCP has a very similar model.

Simple Example

- <https://e6156f20site.s3.us-east-2.amazonaws.com/index.html>



The screenshot shows the AWS S3 console interface for the bucket `e6156f20site`. The left sidebar contains navigation links for Amazon S3, including General purpose buckets, Directory buckets, Table buckets, Vector buckets, Access Grants, Access Points (General Purpose Buckets, FSx file systems), Access Points (Directory Buckets), Object Lambda Access Points, Multi-Region Access Points, Batch Operations, IAM Access Analyzer for S3, Block Public Access settings for this account, Storage Lens, Dashboards, Storage Lens groups, AWS Organizations settings, Feature spotlight, and AWS Marketplace for S3. The main content area shows the bucket's details and a list of 13 objects. The objects are listed in a table with columns for Name, Type, Last modified, Size, and Storage class. The objects include folders and files like `assets/`, `favicon.ico`, `index.html`, `main.js`, `main.js.map`, `polyfills.js`, `polyfills.js.map`, `runtime.js`, `runtime.js.map`, `styles.css`, `styles.css.map`, `vendor.js`, and `vendor.js.map`.

Name	Type	Last modified	Size	Storage class
<code>assets/</code>	Folder	-	-	-
<code>favicon.ico</code>	ico	September 1, 2022, 12:45:21 (UTC-04:00)	948.0 B	Standard
<code>index.html</code>	html	September 1, 2022, 12:45:21 (UTC-04:00)	608.0 B	Standard
<code>main.js</code>	js	September 1, 2022, 12:45:22 (UTC-04:00)	43.6 KB	Standard
<code>main.js.map</code>	map	September 1, 2022, 12:45:22 (UTC-04:00)	22.3 KB	Standard
<code>polyfills.js</code>	js	September 1, 2022, 12:45:22 (UTC-04:00)	149.8 KB	Standard
<code>polyfills.js.map</code>	map	September 1, 2022, 12:45:23 (UTC-04:00)	168.2 KB	Standard
<code>runtime.js</code>	js	September 1, 2022, 12:45:23 (UTC-04:00)	6.2 KB	Standard
<code>runtime.js.map</code>	map	September 1, 2022, 12:45:23 (UTC-04:00)	6.1 KB	Standard
<code>styles.css</code>	css	September 1, 2022, 12:45:24 (UTC-04:00)	143.8 KB	Standard
<code>styles.css.map</code>	map	September 1, 2022, 12:45:24 (UTC-04:00)	761.1 KB	Standard
<code>vendor.js</code>	js	September 1, 2022, 12:45:25 (UTC-04:00)	3.1 MB	Standard
<code>vendor.js.map</code>	map	September 1, 2022, 12:45:27 (UTC-04:00)	3.2 MB	Standard

Could be hierarchical.

I need to migrate to GCP.

Static Content

- You have several options for hosting your static web content on GCP
 - Firebase Hosting – Easiest option, HTTPS + CDN, built-in SPA rewrites, great for static SPAs
 - Cloud Storage + CDN + HTTPS LB – Enterprise-scale, Google-managed SSL, integrates with load balancer infra
 - Cloud Run + NGINX – Flexible, supports custom rewrites and caching headers, serverless scaling
- There are clearly pros and cons to each approach.
- For simplicity, I suggest that you use Firebase.
- Note:
 - I followed ChatGPT's instructions for an hour and nearly had a stroke.
 - I felt like the Sorcerer's Apprentice.
 - I am going to try again over the weekend or switch to Google Cloud Store
- Note 2:
 - I started following the instructions for Google Cloud Storage
 - I nearly has another stroke.
 - I will figure this out over the weekend



**I finally got it to work but do not know how.
The next step is to add Global HTTPS Load Balancer and
custom domain.**

Static Content

- You have several options for hosting your static web content on GCP
 - Firebase Hosting – Easiest option, HTTPS + CDN, built-in SPA rewrites, great for static SPAs
 - Cloud Storage + CDN + HTTPS LB – Enterprise-scale, Google-managed SSL, integrates with load balancer infra
 - Cloud Run + NGINX – Flexible, supports custom rewrites and caching headers, serverless scaling

I managed to get this one to work

[https://storage.googleapis.com/w4153-demo-ui-new/simple_html/index.html/](https://storage.googleapis.com/w4153-demo-ui-new/simple_html/index.html/Users/donald.ferguson/Dropbox/000/000-A-2025-Repox/simple_html)
[Users/donald.ferguson/Dropbox/000/000-A-2025-Repox/simple_html](https://storage.googleapis.com/w4153-demo-ui-new/simple_html/index.html/Users/donald.ferguson/Dropbox/000/000-A-2025-Repox/simple_html)

- NOTE 2:
 - I started following the instructions for Google Cloud Storage
 - I nearly has another stroke.
 - I will figure this out over the weekend

**I finally got it to work but do not know how.
The next step is to add Global HTTPS Load Balancer and custom domain.**

DBaaS

Cloud Concepts – One Perspective

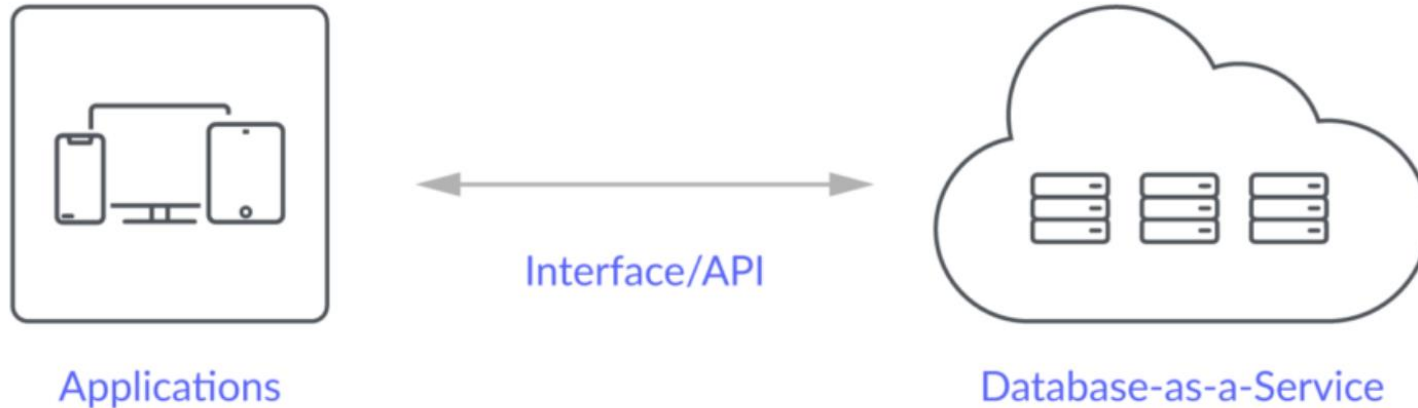
Categorizing and Comparing the Cloud Landscape

<http://www.theenterpriseearchitect.eu/blog/2013/10/12/the-cloud-landscape-described-categorized-and-compared/>

6	SaaS	Applications			End-users
5	App Services	App Services	Communication and Social Services	Data-as-a-Service	Citizen Developers
4	Model-Driven PaaS	Model-Driven aPaaS, bpmPaaS	Model-Driven iPaaS	Data Analytics, baPaaS	Rapid Developers
3	PaaS	aPaaS	iPaaS	dbPaaS	Developers / Coders
2	Foundational PaaS	Application Containers	Routing, Messaging, Orchestration	Object Storage	DevOps
1	Software-Defined Datacenter	Virtual Machines	Software-Defined Networking (SDN), NFV	Software-Defined Storage (SDS), Block Storage	Infrastructure Engineers
0	Hardware	Servers	Switches, Routers	Storage	
		Compute	Communicate	Store	

Database-as-a-Service

“A cloud database is a database that typically runs on a cloud computing platform and access to the database is provided as-a-service. There are two common deployment models: users can run databases on the cloud independently, using a virtual machine image, or they can purchase access to a database service, maintained by a cloud database provider. Of the databases available on the cloud, some are SQL-based and some use a NoSQL data model. Database services take care of scalability and high availability of the database. Database services make the underlying software-stack transparent to the user.” (Wikipedia)



Let's take a look at 3: Relational Data Service, ~~Dynamo DB, MongoDB (Compass)~~

Code

- Show DB on cloud console.
- Show DB via DataGrip
- Explain firewall rule and security implementations
- Explain concept of application in publicly accessible area and data behind another firewall.

12 Factor Applications

12 Factor Applications

<https://dzone.com/articles/12-factor-app-principles-and-cloud-native-microser>



12 Factor App Principles

Codebase

One codebase tracked in revision control, many deploys

Dependencies

Explicitly declare and isolate the dependencies

Config

Store configurations in an environment

Backing Services

Treat backing resources as attached resources

Build, release, and, Run

Strictly separate build and run stages

Processes

Execute the app as one or more stateless processes

Port Binding

Export services via port binding

Concurrency

Scale-out via the process model

Disposability

Maximize the robustness with fast startup and graceful shutdown

Dev/prod parity

Keep development, staging, and production as similar as possible

Logs

Treat logs as event streams

Admin processes

Run admin/management tasks as one-off processes

Dependencies

II. Dependencies

Explicitly declare and isolate dependencies

Most programming languages offer a packaging system for distributing support libraries, such as CPAN for Perl or Rubygems for Ruby. Libraries installed through a packaging system can be installed system-wide (known as “site packages”) or scoped into the directory containing the app (known as “vendoring” or “bundling”).

A **twelve-factor app never relies on implicit existence of system-wide packages**. It declares all dependencies, completely and exactly, via a *dependency declaration* manifest. Furthermore, it uses a *dependency isolation* tool during execution to ensure that no implicit dependencies “leak in” from the surrounding system. The full and explicit dependency specification is applied uniformly to both production and development.

For example, Bundler for Ruby offers the `Gemfile` manifest format for dependency declaration and `bundle exec` for dependency isolation. In Python there are two separate tools for these steps – Pip is used for declaration and Virtualenv for isolation. Even C has Autoconf for dependency declaration, and static linking can provide dependency isolation. No matter what the toolchain, dependency declaration and isolation must always be used together – only one or the other is not sufficient to satisfy twelve-factor.

One benefit of explicit dependency declaration is that it simplifies setup for developers new to the app. The new developer can check out the app’s codebase onto their development machine, requiring only the language runtime and dependency manager installed as prerequisites. They will be able to set up everything needed to run the app’s code with a deterministic *build command*. For example, the build command for Ruby/Bundler is `bundle install`, while for Clojure/Leiningen it is `lein deps`.

Twelve-factor apps also do not rely on the implicit existence of any system tools. Examples include shelling out to ImageMagick or `curl`. While these tools may exist on many or even most systems, there is no guarantee that they will exist on all systems where the app may run in the future, or whether the version found on a future system will be compatible with the app. If the app needs to shell out to a system tool, that tool should be vendored into the app.

- Most application frameworks have a dependency declaration concept.
- I have been showing examples with:
 - Python import
 - requirements.txt
 - etc.
- My examples also show encapsulating libraries and APIs with Python classes.
- II. Dependencies handles “the code,” but we still need to handle the instance. There is a difference between, e.g.
 - pymysql
 - A specific connection.

III. Configurations

III. Config

Store config in the environment

An app's *config* is everything that is likely to vary between deploys (staging, production, developer environments, etc). This includes:

- Resource handles to the database, Memcached, and other backing services
- Credentials to external services such as Amazon S3 or Twitter
- Per-deploy values such as the canonical hostname for the deploy

Apps sometimes store config as constants in the code. This is a violation of twelve-factor, which requires **strict separation of config from code**. Config varies substantially across deploys, code does not.

A litmus test for whether an app has all config correctly factored out of the code is whether the codebase could be made open source at any moment, without compromising any credentials.

Note that this definition of “config” does **not** include internal application config, such as `config/routes.rb` in Rails, or how code modules are connected in Spring. This type of config does not vary between deploys, and so is best done in the code.

Another approach to config is the use of config files which are not checked into revision control, such as `config/database.yml` in Rails. This is a huge improvement over using constants which are checked into the code repo, but still has weaknesses: it's easy to mistakenly check in a config file to the repo; there is a tendency for config files to be scattered about in different places and different formats, making it hard to see and manage all the config in one place. Further, these formats tend to be language- or framework-specific.

The twelve-factor app stores config in *environment variables* (often shortened to *env vars* or *env*). Env vars are easy to change between deploys without changing any code; unlike config files, there is little chance of them being checked into the code repo accidentally; and unlike custom config files, or other config mechanisms such as Java System Properties, they are a language- and OS-agnostic standard.

Another aspect of config management is grouping. Sometimes apps batch config into named groups (often called “environments”) named after specific deploys, such as the `development`, `test`, and `production` environments in Rails. This method does not scale cleanly: as more deploys of the app are created, new environment names are necessary, such as `staging` or `qa`. As the project grows further, developers may add their own special environments like `joes-staging`, resulting in a combinatorial explosion of config which makes managing deploys of the app very brittle.

In a twelve-factor app, env vars are granular controls, each fully orthogonal to other env vars. They are never grouped together as “environments”, but instead are independently managed for each deploy. This is a model that scales up smoothly as the app naturally expands into more deploys over its lifetime.

- Each of the deployment environments we will use has a method for configuring environment variables.
- CI/CD, GitHub actions, etc. can integrate environment variables with the development process.
- Vaults and secrets managers are a better approach for passwords.

IV. Backing Services

IV. Backing services

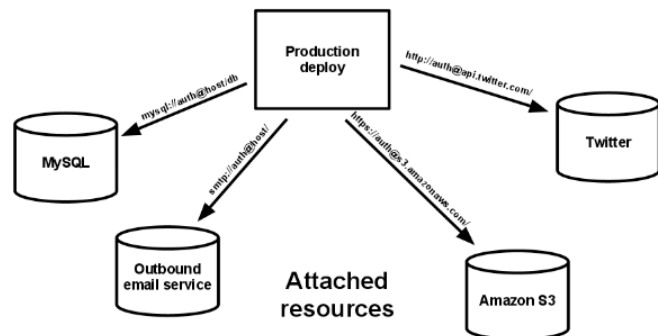
Treat backing services as attached resources

A *backing service* is any service the app consumes over the network as part of its normal operation. Examples include datastores (such as MySQL or CouchDB), messaging/queueing systems (such as RabbitMQ or Beanstalkd), SMTP services for outbound email (such as Postfix), and caching systems (such as Memcached).

Backing services like the database are traditionally managed by the same systems administrators who deploy the app's runtime. In addition to these locally-managed services, the app may also have services provided and managed by third parties. Examples include SMTP services (such as Postmark), metrics-gathering services (such as New Relic or Loggly), binary asset services (such as Amazon S3), and even API-accessible consumer services (such as Twitter, Google Maps, or Last.fm).

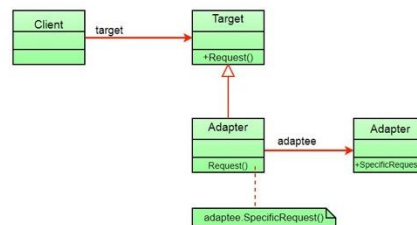
The code for a twelve-factor app makes no distinction between local and third party services. To the app, both are attached resources, accessed via a URL or other locator/credentials stored in the `config`. A deploy of the twelve-factor app should be able to swap out a local MySQL database with one managed by a third party (such as Amazon RDS) without any changes to the app's code. Likewise, a local SMTP server could be swapped with a third-party SMTP service (such as Postmark) without code changes. In both cases, only the resource handle in the config needs to change.

Each distinct backing service is a *resource*. For example, a MySQL database is a resource; two MySQL databases (used for sharding at the application layer) qualify as two distinct resources. The twelve-factor app treats these databases as *attached resources*, which indicates their loose coupling to the deploy they are attached to.



Resources can be attached to and detached from deploys at will. For example, if the app's database is misbehaving due to a hardware issue, the app's administrator might spin up a new database server restored from a recent backup. The current production database could be detached, and the new database attached – all without any code changes.

- My code has shown encapsulating backing services with a shallow adaptor.
- This is an example of the *adaptor pattern*.



- You seen examples of things like `AbstractBaseDataService`,
- The approach is also an application of several aspects of SOLID.
(<https://en.wikipedia.org/wiki/SOLID>)

Next Sprint

Next Sprint

- Well-design REST applications for basic and composite microservices, e.g.
 - eTag
 - Query parameters
 - Linked data and relative paths
 - 201 Created for POST
 - One deployed on Cloud Compute and one deployed using CaaS on Cloud Run
- Composite microservices
 - Well-defined API
 - One operation implement synchronously and one operation asynchronously
- DBaaS and one microservice/database pattern
 - 1 SQL VM/Cloud Compute MySQL instance
 - 1 or two Cloud SQL instances
- Simple Cloud Storage based web UI accessing composite, and other microservices
- Template/scaffolded Google Cloud Function triggered by pub/sub
- Updated GitHub projects and Trello for sprints